Embodied AI — Comprehensive Concept & Quiz Fact Sheet

Lectures 2–12: Core Intuitions, Trade-offs, Pipelines & Key Equations

A.Y. 2026/27 · High-Yield Review

Lecture 2: 3D Representations

Core Concept: In 2D, pixel grids are universal. In 3D, there is *no single best representation*—each is a compromise between acquisition, memory, rendering speed, topology flexibility, and neural network compatibility.

- **Manifold:** A space that locally resembles flat Euclidean space $\mathbb{R}^n$ (e.g., an ant on a sphere sees a flat 2D plane). A 2-manifold is a **surface**. Non-manifold mesh errors occur when an edge is shared by > 2 faces or a vertex links two disconnected fans.
- **2.5D Representations (Image-like):**
  - **Depth Map ($D[x_s] \in \mathbb{R}^+$) & Normal Map ($N[x_s] \in \mathbb{S}^2$):** Store distance along the optical axis (or surface normal) per pixel. Fast and CNN-friendly, but **incomplete** (only captures the first visible hit; blind to occluded geometry).
  - **Layered Depth Images (LDI):** Store a list of multiple hits along each ray to handle occlusion, but layers jump discontinuously across pixels.
  - **Multi-Plane Images (MPI):** Stack of fronto-parallel (RGB, α) planes at fixed depths; fast to render for small view shifts, poor resolution far away.
- **Boundary / Surface Representations (Track the "skin", fix topology):**
  - **Point Clouds ($\{x_i\}_{i=1}^N \in \mathbb{R}^{N\times 3}$):** Unordered set of 3D points. Networks *must be permutation invariant* (use symmetric functions like $\max$, mean, sum; never flatten into an FC layer). **No explicit connectivity** (cannot tell a closed loop from a gapped loop without huge point density). **Surfels** add normals n_i and radii to render as small flat disks.
  - **Triangle Meshes ($V \in \mathbb{R}^{N\times 3}, F \in \mathbb{Z}^{M\times 3}$):** Piecewise linear surfaces. Separate **geometry (V, float, differentiable)** from **topology (F, int, discrete/combinatorial)**. Gradient descent can move vertices V, but cannot easily change F or change genus (e.g., torus $\leftrightarrow$ sphere).
  - **Mesh Properties:** Winding order (a, b, c) sets the outward normal $n = (b - a) \times (c - a)$; swapping two indices flips the normal. For closed genus-0 meshes, Euler's formula ($V - E + F = 2$ with $E = 3F/2$) yields $F = 2V - 4 \approx 2V$. Ray-triangle intersection uses **barycentric coordinates** $p = \alpha a + \beta b + \gamma c$ ($\alpha + \beta + \gamma = 1, \geq 0$), which also interpolate UV textures.
  - **Parametric Surfaces ($f(u) = x \in \mathbb{R}^3, u \in \mathcal{M} \subset \mathbb{R}^2$):** E.g., Bézier patches. *Easy to sample/generate points* (plug in u), *hard to test inside/outside or intersect rays*, and topology is locked to the domain $\mathcal{M}$.
- **Volumetric Representations (Assign a value to every 3D point; allow arbitrary topology changes):**
  - **Voxel Grids ($V[i, j, k] \in [0, 1]$):** Fixed-size 3D arrays compatible with 3D CNNs and arbitrary topology. **Flaw:** Memory grows cubically as $\mathcal{O}(n^3)$ while the surface only occupies $\mathcal{O}(n^2)$ cells (pays for empty air). **Octrees** subdivide adaptively near boundaries, saving memory at the cost of irregular tree structures.
  - **Implicit Surfaces ($\mathcal{S} = \{x \in \mathbb{R}^3 \mid f(x) = 0\}$):** Convention: $f(x) < 0$ inside, $f(x) > 0$ outside. **Mantra: Parametric generates, Implicit tests.** For f to define a valid surface, it must change sign across $\mathcal{S}$ and have $\nabla f \neq 0$ on $\mathcal{S}$.
  - **Implicit Perks:** 1) **Normals for free:** $n(x) = \nabla f(x)/\|\nabla f(x)\|$ via autodiff. 2) **CSG Boolean operations:** Union $A \cup B \Rightarrow \min(f_A, f_B)$; Intersection $A \cap B \Rightarrow \max(f_A, f_B)$; Subtraction $A \setminus B \Rightarrow \max(f_A, -f_B)$.
  - **Signed Distance Function (SDF):** An implicit field where $|f(x)|$ is the exact Euclidean distance to the closest surface point. Satisfies the **Eikonal equation** $\|\nabla f(x)\| = 1$. Enables **Sphere Tracing**: stepping along a ray by $t \leftarrow t + f(o + td)$ is guaranteed never to overshoot the surface because the ball of radius $|f(x)|$ is empty. Stored on grids (**TSDF** truncates to $[-\tau, +\tau]$ near surfaces) or in MLPs.
  - **Marching Cubes:** Converts an implicit field to a triangle mesh by evaluating f on a grid, finding edges where corners have opposite signs, and linearly interpolating vertex crossings $t = \frac{f(a)}{f(a)-f(b)}$.
  - **Density Fields ($\sigma(x) \geq 0$):** Represent absorption per unit length for semi-transparent media (smoke, CT scans, NeRF, 3D Gaussians) with no hard surface boundary.

Representation	Sample Points	Inside/Outside	Change Topology	Direct Render	Memory Scaling
Point Cloud	Given	Hard (no surface)	Any point set	Splats	$\mathcal{O}(N)$ points
Triangle Mesh	Easy (barycentric)	Ray crossings	Hard (fixed F)	Fast GPU raster	$\mathcal{O}(V+F)$
Parametric $f(u)$	Easy (plug in u)	Hard (invert f)	Locked to $\mathcal{M}$	Convert to mesh	Control points
Voxel Grid	Marching cubes	Easy (cell lookup)	Easy (any)	Blocky / march	$\mathcal{O}(n^3)$ (wasteful)
Implicit / SDF	Hard (March. cubes)	Trivial (sign(f))	Easy (continuous)	Sphere tracing	Grid $\mathcal{O}(n^3)$ or MLP

Lecture 3: Depth Measurement, Priors & Diffusion (Marigold)

Core Concept: Pinhole projection $p^c = zK^{-1}\tilde{x}_s$ destroys distance z. A single image leaves global scale unobservable (a small near object looks identical to a large far object). Every depth method must **pay with a currency**: *Time*, *Baseline*, *Computation/Motion*, or a *Learned Prior*.

- **The Ambiguity Ladder (Free Parameters):**
  - **Metric Depth (0 free params):** True meters (from LiDAR or calibrated stereo).
  - **Up to Scale (1 free param, s):** $\hat{D} \approx sD^*$. Shape is preserved (similarity transform). Produced by monocular SfM or Eigen's log-space loss. Pinned down by 1 known length.
  - **Affine-Invariant Depth/Disparity (2 free params, s,t):** $\hat{q} \approx sq^* + t$. Pinned down by 2 known depths.
  - **Ordinal (monotone map):** Only relative front/back ordering is known.
- **Physical Measurement Principles & Error Laws:**
  - **Time of Flight (ToF / LiDAR):** Active only (needs known departure time). Measures round-trip delay $r = \frac{ct}{2}$. Timing error floor $\delta r = \frac{c\delta t}{2}$ is **independent of range** (1 ns $\approx$ 15 cm). **Flaws:** Structured sparsity (vertical scan-line gap grows linearly with range $s(r) \approx r\Delta\theta_{\text{el}}$, missing distant pedestrians), missing returns $\neq$ zero depth, and "flying points" at silhouettes where one beam straddles foreground and background.
  - **Projecting LiDAR to Depth Image:** Transform via extrinsics $p^c = R_{cl}p^l + t_{cl}$, project $\tilde{x}_s = \pi(Kp^c)$, use a z-buffer (keep smallest $p^c_z > 0$). Note: Depth map records camera z-axis $z = r\cos\theta$, *not* radial range $r = \|p^c\|$.
  - **Triangulation (Stereo):** $z = \frac{fb}{d}$ where baseline b provides metric scale. Depth error grows **quadratically with depth**: $\delta z \approx \frac{z^2}{fb}\delta d$. (LiDAR is precise at long range but sparse; stereo is dense at short range but degrades quadratically).
- **Closed-Form Monocular Cues & Network Shortcuts:**
  - **Vanishing Points:** $\lambda v = KRd$ (depends only on rotation/pitch/roll, independent of translation).
  - **Apparent Size:** $z = \frac{fH}{h}$ (requires knowing real object height H).
  - **Vertical Ground Contact Position:** $z = \frac{fY}{y_s - y_h}$ (requires flat ground, camera height Y, and horizon row y_h; error grows as z^2).
  - **What Networks Actually Use (Probes):** 1) *Pasted-Car Experiment:* Networks rely on **vertical image position** over apparent size. 2) *Pitch/Roll Crops:* Networks read raw pixel row rather than true distance to the horizon. 3) *Camera Focal Length Bias:* Networks memorize training focal length f_0, predicting $\hat{z} = \frac{f_0}{f}z$ on test camera f. 4) *Anamorphic Street Art:* Flat pavement painted like a hole fools single-image networks with zero uncertainty warning.
- **Supervised & Self-Supervised Single-View Depth:**
  - **Eigen Scale-Invariant Loss:** Log-MSE minus the per-image mean offset $\alpha = \frac{1}{n}\sum(\log y_i^* - \log y_i)$, which equals the variance $\text{Var}(d_i)$ of log-differences. Invariant to global scaling $y \mapsto sy$.
  - **Affine Invariance in Disparity vs. Depth:** In disparity ($\frac{1}{z} \mapsto \frac{s}{z} + t \iff z \mapsto \frac{z}{s+tz}$), back-projecting yields a projective map $\hat{p} = \frac{p}{s+tp_z}$ that **keeps planes flat** (tilts them without bending). In depth ($\hat{z} = sz + t$), back-projecting gives $\hat{p} = (s + \frac{t}{z})p$: scale s preserves shape, but **shift** t **distorts/bends 3D shapes** because rays fan out.
  - **MiDaS:** Mixes heterogeneous datasets (metric, SfM, stereo movies) by predicting in **affine-invariant disparity space**, aligning each training image via closed-form least squares (s,t) before computing loss + multi-scale gradient loss.
  - **Depth Anything V2:** Trains a large teacher on exact synthetic data (sharp edges, no sensor holes), uses the teacher to pseudo-label millions of unlabeled real images, and trains a student on those pseudo-labels.
  - **Self-Supervised Stereo:** Trains without depth labels using photometric warp loss $\mathcal{L}_{\text{photo}} = \|\tilde{I}_L - I_L\|$ via **backward warping** $\tilde{I}_L(u,v) = I_R(u - d(u,v), v)$ (avoids holes/collisions of forward warping). **Blind spots:** Textureless regions (image gradient $I'_R = 0 \Rightarrow \frac{\partial \mathcal{L}}{\partial d} = 0$) and occlusions.
- **Diffusion Priors, Marigold & Marigold-DC:**
  - **Regression vs. Generative:** MSE regression outputs the conditional mean $\mathbb{E}[Y|x]$, which averages foreground and background at depth edges (creating blurry flying pixels). Diffusion samples a sharp mode from $p(D|I)$.
  - **Denoiser as a Prior:** Trained on $L_{\text{simple}} = \mathbb{E}\|\epsilon_\theta(x_0 + \sigma\epsilon, \sigma) - \epsilon\|^2$. The ideal denoiser satisfies $\sigma\epsilon^*(x,\sigma) = \nabla_x D_\sigma(x)$ (the gradient of the **smoothed distance function** to the data manifold): at small σ it points to the nearest data point; at large σ to the dataset center.
  - **DDIM Step:** $x_{t-1} = x_t + (\sigma_{t-1} - \sigma_t)\epsilon_\theta(x_t, \sigma_t)$, walking toward the clean Tweedie preview $\hat{x}_0^t = x_t - \sigma_t\epsilon_\theta(x_t, \sigma_t)$.
  - **Marigold:** Repurposes pretrained **Stable Diffusion**: freezes the VAE, encodes image $z^{(I)}$ and percentile-normalized depth $\tilde{D} \in [-1,1]$ into latent space, concatenates $[z_\sigma^{(D)}, z^{(I)}]$ (doubling first U-Net layer channels and halving weights),

and fine-tunes on 74K synthetic pairs. At test time, ensembles $N = 10$ runs (50 steps each $= 500$ evals) by mutually aligning scale/shift and taking the pixel median. **Output:** Affine-invariant depth (2 free params, slow).
  - **Marigold-DC (Depth Completion via Test-Time Guidance):** Upgrades sparse metric LiDAR measurements c to a dense metric depth map **with zero retraining**. Inside the 50-step DDIM loop, decodes the Tweedie preview $\hat{z}_0^t$, applies learnable scale/shift $\hat{d}_t^{(m)} = \hat{a}\mathcal{D}(\hat{z}_0^t) + \hat{b}$, computes error against sparse LiDAR pixels, and takes a gradient descent step on $(z_t, \hat{a}, \hat{b})$ while keeping U-Net/VAE weights frozen.

Lecture 4: Stereo Depth Estimation (Classical to Uncalibrated 3D)

Core Concept: Calibration and rectification turn 2D correspondence search into a 1D horizontal scanline search. Modern 3D pointmap networks (DUSt3R/MASt3R) drop calibration and rectification entirely by matching in 3D space.

- **Classical Stereo Pipeline:** 1) Calibrate → 2) Rectify → 3) Compute disparity → 4) Left-Right consistency outlier filter → 5) Triangulate $z = fb/d$ → 6) Fuse 3D model.
- **Image Rectification:** Applies a pure rotation R_{rect} about each camera center so both image planes become coplanar and parallel to the baseline b. Epipoles move to infinity along the horizontal axis, and the epipolar constraint reduces to $y_1 = y_2$ (1D search along the same row). **Fails** under forward motion (epipole lands inside the image).
- **Block Matching & Failure Modes:**
  - **Metrics: SSD** $\|w_L - w_R\|_2^2$ (sensitive to exposure); **ZNCC** $\frac{(w_L-\mu_L)^\top(w_R-\mu_R)}{\|w_L-\mu_L\|\|w_R-\mu_R\|}$ (invariant to linear gain and bias $aw + b$).
  - **Window Size Trade-off:** Small windows → noisy, ambiguous matches. Large windows → smooth, but violate the fronto-parallel assumption on slanted surfaces and cause **border bleeding** (foreground edges fatten over weakly textured backgrounds).
  - **Half-Occlusions:** A strip of background of width $d_f - d_b$ immediately to the left of a foreground object in the left image is hidden in the right image. Caught by the **Left-Right Consistency Check**: $|d_L(x, y) - d_R(x - d_L(x, y), y)| \le \tau$.
- **Learned Stereo Evolution:**
  - **Siamese Networks (Zbontar & LeCun):** Patch-level CNN trained on triplets (anchor, positive within ± 1 px, hard negative 3–6 px away) using **Hinge Loss** $\max(0, m + s_- - s_+)$. *Cosine-similarity head* (dot product of normalized features) is much faster than an MLP similarity head.
  - **DispNet:** End-to-end 2D encoder-decoder with a 1D correlation layer; pretrained on synthetic data (FlyingThings3D).
  - **GC-Net:** Builds a 4D cost volume ($H \times W \times D \times F$), regularizes with 3D convolutions, and extracts sub-pixel disparity via differentiable **Soft Argmin**: $\hat{d} = \sum_{d=0}^{d_{\max}} d \cdot \text{softmax}(-c_\theta(d))$. **Flaw:** High 3D memory cost, and bimodal cost curves at occlusion edges average out to flying pixels between modes.
- **Stereo Without Calibration: DUSt3R & MASt3R:**
  - **Pointmap ($X^{a,1} \in \mathbb{R}^{H\times W\times 3}$):** Assigns every pixel of image $a \in \{1, 2\}$ its 3D coordinate **expressed in camera 1's coordinate frame**. Two pixels correspond iff their 3D coordinates coincide.
  - **Architecture:** Shared ViT-Large Siamese encoder → twin ViT-Base decoders coupled via **cross-attention** → regression heads outputting pointmaps $\hat{X}^{a,1}$ and confidence maps C^a.
  - **Confidence-Weighted Loss:** $\mathcal{L}_{\text{conf}} = \sum C^a[i] \left\| \frac{1}{\hat{\rho}}\hat{X}^{a,1}[i] - \frac{1}{\rho}X^{a,1}[i] \right\| - \alpha \log C^a[i]$. Dividing by mean distances $\rho, \hat{\rho}$ normalizes unknown scale; the $-\alpha \log C$ penalty prevents the network from cheating by setting $C \to 0$ everywhere (optimal confidence is $C^* = \alpha/\ell_{\text{regr}}$).
  - **MASt3R Additions:** Because regressed 3D points are robust to huge viewpoint changes but locally imprecise, MASt3R adds a **local descriptor head** ($F^a \in \mathbb{R}^{H\times W\times 24}$) trained with **InfoNCE loss** (a smooth multi-class generalization of hinge loss) and extracts matches via 2D **reciprocal nearest neighbors** (mutual NN across the whole image).
  - **Trade-off vs. Classical Stereo:** Handles uncalibrated cameras, wide baselines, and forward motion, but sacrifices measured metric scale (scale comes from a learned prior), high image resolution, and low compute.

Lecture 5: Structure-from-Motion (Classical COLMAP vs. Feed-Forward VGGT)

Core Concept: Classical SfM alternates feature matching, minimal geometric solvers inside RANSAC, and sparse nonlinear Bundle Adjustment (BA). Feed-forward transformers (VGGT) predict cameras, depth, pointmaps, and tracks for dozens of views in one pass (< 1 s), providing an ideal initialization for BA.

- **Camera Calibration (Zhang's Method):** Captures a flat checkerboard ($Z = 0$) in multiple poses. Each view defines a plane-to-image homography $H = \lambda K[r_1\ r_2\ t]$. Orthonormality of rotation columns ($r_1^\top r_2 = 0, \|r_1\| = \|r_2\|$) yields 2

linear constraints per view on $B = K^{-\top}K^{-1}$ (5 DoF $\Rightarrow \geq 3$ tilted views needed), solved via SVD and refined jointly with lens distortion via nonlinear least squares (reprojection error).

- **Feature Detection & Matching:**
  - **SIFT:** Detects scale-space extrema in **Difference of Gaussians (DoG)** (approximates $\sigma^2\nabla^2 G$ for scale invariance), rotates patch to dominant gradient orientation (rotation invariance), and pools 4×4 cells of 8-bin gradient histograms into a **128D vector**. Filtered via **Lowe's Ratio Test**: $\frac{\|f - f'_{(1)}\|}{\|f - f'_{(2)}\|} < 0.8$ (rejects ambiguous repetitive patterns).
  - **SuperPoint:** Self-supervised CNN with shared encoder ($8\times$ downsampling) and two heads: keypoint head ($H/8 \times W/8 \times 65$: 64 pixel bins + 1 dustbin, softmax) and descriptor head (256D). Trained via 1) MagicPoint on synthetic shapes $\rightarrow$ 2) **Homographic Adaptation** (averages detections across $N_h = 100$ random homography warps of real images to enforce covariance) $\rightarrow$ 3) joint training on warped pairs. SIFT still wins at finest sub-pixel localization ($\varepsilon = 1$); SuperPoint wins on descriptor mAP and speed (~ 70 fps).
- **Two-Frame SfM & Epipolar Geometry:**
  - **Essential Matrix ($E = [t]_\times R$):** Operates on calibrated rays $\tilde{x}_i = K_i^{-1}\bar{x}_i$ via $\tilde{x}_2^\top E \tilde{x}_1 = 0$. Has **rank 2**, **5 DoF** (3 rot + 3 trans $-$ 1 scale), and singular values $(\sigma, \sigma, 0)$. Estimated via the **normalized 8-point algorithm** (SVD + projecting singular values to $(1, 1, 0)$) or minimal **5-point solver** inside RANSAC.
  - **4-Fold Decomposition & Cheirality:** Decomposing $E = U\,\mathrm{diag}(1, 1, 0)V^\top$ yields 4 pose candidates: ($UWV^\top$ or $UW^\top V^\top$) and $t = \pm u_3$ (baseline sign flip + $180°$ twisted pair rotation). Only **1 solution** puts triangulated points in front of *both* cameras ($z_1 > 0, z_2 > 0$, **cheirality check**).
  - **Fundamental Matrix ($F = K_2^{-\top} E K_1^{-1}$):** Used when intrinsics K_i are unknown ($\bar{x}_2^\top F \bar{x}_1 = 0$); rank 2, **7 DoF**.
  - **Triangulation & Parallax Trade-off:** Linear DLT ($\bar{x}_i \times P_i \tilde{x}_w = 0$) or nonlinear reprojection minimization. **Trade-off:** Small baseline/parallax angle γ makes feature matching easy, but rays are nearly parallel so depth uncertainty explodes ($\sigma_Z \propto 1/\gamma$); wide baseline gives accurate triangulation but hard matching.
- **Bundle Adjustment (BA) & Incremental SfM (COLMAP):**
  - **Formulation:** Jointly minimizes reprojection error $\sum_{i,p} w_{ip} \|x_{ip}^s - \pi(K_i(R_i x_p^w + t_i))\|_2^2$ over all camera poses $T_i \in SE(3)$ and 3D points $x_p^w \in \mathbb{R}^3$.
  - **Gauge Freedom (7 DoF):** The cost is invariant to global rigid motion (6 DoF) and global scale (1 DoF), giving the Hessian a 7D null space (fixed by anchoring $T_1 = I$ and $\|t_2\| = 1$).
  - **Lie Algebra Updates:** Rotations are not a vector space; poses are updated in the tangent space $\mathfrak{se}(3) \cong \mathbb{R}^6$ via the exponential map $T_i \leftarrow \mathrm{Exp}(\Delta\xi_i) T_i$.
  - **Schur Complement Trick:** In the Gauss-Newton normal equations $H\Delta s = -b$, points never talk directly to other points, so H_{xx} is block-diagonal (3×3 blocks). Eliminating points yields the **reduced camera system** $(H_{\xi\xi} - H_{\xi x} H_{xx}^{-1} H_{x\xi})\Delta\xi = -(b_\xi - H_{\xi x} H_{xx}^{-1} b_x)$ of size $6N$ instead of $6N + 3P$ ($\sim 10^8\times$ speedup).
  - **Incremental SfM Loop:** Seed with a wide-baseline two-view reconstruction $\rightarrow$ register next image via **PnP / P3P** (2D-to-3D matches) $\rightarrow$ triangulate new points $\rightarrow$ run local BA (and periodic global BA) + filter outliers. **Flaw:** Slow, sequential error accumulation/drift, fails on low overlap or textureless scenes. (GLOMAP is a non-learned global alternative).
- **Feed-Forward SfM: VGGT (Visual Geometry Grounded Transformer):**
  - **Input/Output:** Takes N uncalibrated images; outputs per-image camera parameters $\hat{g}_i \in \mathbb{R}^9$ (quaternion, translation, FoV), dense depth $\hat{D}_i$, world-frame pointmaps $\hat{X}_i$ (in camera 1's frame), and tracking features F_i **in one forward pass (< 0.2 s)**.
  - **Architecture:** DINO patch tokens + 1 learned **camera token** per image $\rightarrow L = 24$ layers of **Alternating Attention** (alternating *Global Attention* across all tokens of all N images with $N\times$ cheaper *Frame Attention* within each single image) $\rightarrow$ Camera head + DPT dense head + CoTracker2 tracking head.
  - **Redundancy & BA Synergy:** Predicting redundant outputs (depth + cameras + pointmaps) improves accuracy. Using VGGT predictions to initialize classical Bundle Adjustment eliminates incremental initialization and boosts pose accuracy from $85.3 \rightarrow 93.5$ AUC@30 in ~ 1.8 s.

Lecture 6: Visual SLAM (Filtering, Factor Graphs, ORB-SLAM & Active SLAM)

Core Concept: SLAM is joint estimation of trajectory and map in real time, with loop closure to eliminate dead-reckoning drift ($\sigma\sqrt{t}$). Modern SLAM drops dense EKF filtering in favor of sparse keyframe factor graphs.

- **Filtering (Online SLAM: $p(x_t, m \mid z_{1:t}, u_{1:t})$) vs. Graph-Based (Full SLAM: $p(x_{1:t}, m \mid z_{1:t}, u_{1:t})$):**
  - **Bayes Filter / EKF-SLAM (MonoSLAM):** Maintains a single joint Gaussian $\mathcal{N}(\mu, \Sigma)$ over the current camera state (13D) and N 3D landmarks ($3N$). Prediction touches only camera blocks, but the **Kalman update touches the entire covariance matrix**: $\mathcal{O}(N^2)$ time and memory per frame, limiting maps to ~ 100 points at 30 Hz.
  - **Why Marginalizing Past Poses Hurts:** Marginalizing out a past pose x_k via the Schur complement connects every

landmark seen by x_k to every other landmark it saw (**fill-in**, turning a sparse star into a dense clique). Also, EKF linearizes Jacobians only once, risking divergence.
  - **Factor Graphs & Pose Graphs:** Keep keyframe poses in the state! Variable nodes (poses, points) connect to factor nodes (priors, odometry, reprojection, loop closures). Maximizing the posterior product of Gaussian factors equals **nonlinear least squares**: $\min \sum_k e_k^\top \Omega_k e_k$, where $\Omega_k = \Sigma_k^{-1}$ is the information matrix.
  - **Sparsity of H:** In Gauss-Newton/Levenberg-Marquardt ($H\Delta x = -b$), block H_{ij} is non-zero **if and only if an edge links variables i and j** (graph adjacency matrix). Keeping keyframes keeps H sparse!
- **The ORB-SLAM Pipeline (3 Parallel Threads):**
  - **Features:** Uses multi-scale **ORB** (oriented FAST corners + binary descriptors) matched via fast bitwise XOR **Hamming distance** (< 33 ms, unlike SIFT).
  - **Map Initialization:** Runs RANSAC in parallel for a **Homography** H_{21} (planar/low-parallax) and **Fundamental matrix** F_{21} (general scene); picks H if $S_H/(S_H + S_F) > 0.45$, and waits to initialize until sufficient parallax exists.
  - **Thread 1 — Tracking (every frame, real-time):** Predicts pose via constant-velocity model, projects local map points from covisible keyframes, and runs **motion-only BA** (optimizing only the 6-DoF current pose with Huber loss). Inserts keyframes generously.
  - **Thread 2 — Local Mapping (per keyframe):** Updates the **Covisibility Graph** (edges between keyframes sharing ≥ 15 points), triangulates new points, runs **Local BA** on the covisible neighborhood (keeping outer keyframes fixed), and culls redundant keyframes (where $\geq 90\%$ of points are seen by ≥ 3 other keyframes).
  - **Thread 3 — Loop Closing & Relocalization:** Queries a hierarchical **Bag of Binary Words (DBoW2)** vocabulary tree (k branches, L levels $\Rightarrow k^L$ words in only kL comparisons) with an **inverted index**. Because monocular SLAM has **7-DoF drift** (rotation, translation, *and scale*), loop correction optimizes the Essential Graph over **Sim**(3) **similarity transforms** ($e_{ij} = \log(Z_{ij}^{-1} S_i S_j^{-1})^\vee \in \mathbb{R}^7$) rather than $SE(3)$.
- **Advanced SLAM Directions:**
  - **Rolling-Shutter / Continuous-Time SLAM:** Rolling-shutter cameras expose each scanline at a different timestamp, skewing images under motion. Representing the trajectory as a continuous **cumulative cubic B-spline on** $SE(3)$ lets every scanline (or IMU reading) query its exact timestamp t, cutting loop error by $8\times$.
  - **Active SLAM & FisherRF:** Balances **exploration** (new frontiers) vs. **exploitation** (revisiting known areas to close loops and shrink covariance). Minimizes uncertainty volume via **D-optimality** $\det(\Sigma)^{1/n}$. In radiance fields (**FisherRF**), selects the next view x maximizing expected information gain $\mathrm{IG}(x) \leq \frac{1}{2}\mathrm{tr}(I_x I_{\mathcal{D}}^{-1})$ using the diagonal Fisher matrix $I_x = J_x^\top J_x$ (evaluated without needing a ground-truth image).
  - **Learned Front Ends: MASt3R-SLAM** uses Sim(3) pose graphs; **VGGT-SLAM** aligns uncalibrated submaps using $SL(4)$ **projective homographies (15 DoF)**.

Lectures 7 & 8: Volume Rendering, NeRF, Plenoxels & Instant-NGP

Core Concept: Drop the hard-surface assumption and model the scene as a continuous field of density $\sigma(x)$ and view-dependent emitted radiance $c(x, \omega)$, trained via differentiable ray marching against posed 2D images.

- **Radiative Transfer vs. Emission-Absorption (EA) Model (Lecture 7):**
  - **4 Physical Interactions:** Loss from **Absorption** $(-\sigma_a L)$ and **Out-scattering** $(-\sigma_s L)$; Gain from **Emission** $(+\sigma_a L_e)$ and **In-scattering** $(+\sigma_s L_s$, which integrates incoming light over the whole sphere of directions recursively).
  - **Why NeRF Drops Scattering:** In-scattering is recursive and expensive. By allowing an arbitrary view-dependent effective emission $L'_e(x, \omega) = L_e + \frac{\sigma_s}{\sigma_a}(L_s - L)$, the **Emission-Absorption (EA) model** $dL = -\sigma(x) L\, dz + \sigma(x) L_e(x, \omega)\, dz$ can bake scattered/reflected appearance directly into $L_e(x, \omega)$ (at the cost of not being physically relightable).
  - **Transmittance & Continuous Volume Rendering Integral:**

$$T(x, x_t) = \exp\left(-\int_0^t \sigma(x - s\omega)\, ds\right), \qquad L(x, \omega) = \int_0^z T(x, x_t)\, \sigma(x_t)\, L_e(x_t, \omega)\, dt + T(x, x_z) L_{\text{bg}}$$

    Transmittance is symmetric ($T(x, y) = T(y, x)$) and multiplicative ($T(x, y) = T(x, z)T(z, y)$).
  - **Computational Ray Marching (Alpha Compositing):** Discretize the ray into N homogeneous segments of length Δt_i:

$$L(x, \omega) \approx \sum_{i=1}^{N} w_i c_i + T_{N+1} L_{\text{bg}}, \qquad w_i = T_i \alpha_i, \qquad \alpha_i = 1 - e^{-\sigma_i \Delta t_i}, \qquad T_i = \prod_{j<i} (1 - \alpha_j)$$

    Because $w_i = T_i - T_{i+1}$, the weights telescope: $\sum_{i=1}^{N} w_i + T_{N+1} = 1$ (a partition of unity; T_{N+1} is the background mask).
- **Neural Radiance Fields (NeRF, Lecture 8):**
  - **Architecture Inductive Bias:** An MLP $f_\theta : (\gamma(x), \gamma(\omega)) \mapsto (\sigma, c)$. Position x enters at layer 1 to output density $\sigma(x)$; viewing direction ω is injected **only in the final head layers** so that **geometry** $\sigma(x)$ **is strictly view-independent** while color $c(x, \omega)$ captures view-dependent specularities.

- **Positional Encoding (Overcoming Spectral Bias):** Standard ReLU MLPs are biased toward low-frequency smooth functions, producing blurry renders. Lifting inputs via $\gamma(v) = [\sin(2^k\pi v), \cos(2^k\pi v)]_{k=0}^{L-1}$ ($L = 10$ for x, $L = 4$ for ω) induces a tunable high-frequency stationary kernel $\langle\gamma(v), \gamma(v')\rangle = \sum \cos(2^k(v - v'))$ so nearby points can take sharp, distinct values.
- **Hierarchical (Coarse-to-Fine) Sampling:** Uniform sampling wastes MLP evaluations in empty air or occluded regions. A **coarse pass** evaluates $N_c = 64$ uniform samples, normalizes the rendering weights $\hat{w}_i = w_i / \sum w_j$ into a piecewise-constant PDF along the ray, and uses **inverse-CDF transform sampling** to concentrate $N_f = 128$ **fine samples** right around visible surfaces where w_i peaks.
- **Inverse-Depth / NDC Sampling:** For unbounded forward-facing scenes, sampling linearly in $1/z$ (or mapping the frustum to Normalized Device Coordinates $[-1, 1]^3$) allocates high resolution to close objects (large parallax) and coarse resolution to distant backgrounds.
- **Anti-Aliasing (mip-NeRF):** Casting a single 1D ray per pixel aliases when camera distance or resolution changes. **mip-NeRF** casts a 3D **conical frustum**, approximates each frustum slice with a 3D Gaussian $\mathcal{N}(\mu, \Sigma)$, and feeds the **Integrated Positional Encoding (IPE)** $\mathbb{E}[\sin(2^k X)] = \sin(2^k\mu)\exp(-\frac{1}{2}4^k s^2)$, which automatically damps high frequencies whose wavelength is smaller than the pixel footprint s.

- **Trading MLP Compute for Grid Lookups (Plenoxels & Instant-NGP):**
  - **Plenoxels (No Neural Network!):** Stores density σ and **Spherical Harmonics (SH) coefficients** ($\ell_{\max} = 2 \Rightarrow 9 \times 3 = 27$ coeffs $+ 1\,\sigma = 28$ values/voxel) on a **sparse voxel grid**. Queries via trilinear interpolation, renders via ray marching, and optimizes voxel values directly with photometric loss + Total Variation ($\mathcal{L}_{\text{TV}}$) regularization, pruning empty voxels. Trains in minutes instead of hours.
  - **Instant-NGP (Multiresolution Hash Encoding):** Uses $L = 16$ grid resolutions N_ℓ. Instead of storing N_ℓ^3 cells, maps integer cell corners to a fixed-size table of T feature vectors ($F = 2$ dims) via a spatial XOR hash $h(c) = (\bigoplus c_i\pi_i) \bmod T$, interpolates across the 8 corners at each level, concatenates all L levels with $\gamma(\omega)$, and feeds a **tiny 2-layer MLP**. Memory is bounded by $L \cdot T \cdot F$ regardless of spatial resolution; hash collisions are resolved automatically by gradient descent because surface gradients dominate empty space. Trains in **seconds**.

Lecture 9: Neural Implicit Rendering (IDR, VolSDF & NeuS)

Core Concept: NeRF's unconstrained density produces noisy, bumpy surfaces. Enforcing that geometry is the zero level set of an SDF ($f_\theta(x) = 0, \|\nabla f_\theta\| = 1$) gives clean surfaces, rendered either via single-point ray intersection (IDR) or unbiased SDF-to-density volume rendering (VolSDF, NeuS).

- **Pure Surface Rendering (IDR — Yariv et al.):**
  - **Step 1 (Ray-Surface Intersection):** Find the root t^* where $\bar{f}_\theta(t^*) = f_\theta(x_0 + t^*\omega) = 0$ using a hybrid of **sphere tracing** and the **secant method** (since neural f_θ is only approximately an SDF and can overshoot).
  - **Step 2 (Analytic Implicit Differentiation):** We *do not* backpropagate through the iterative sphere-tracing loop! Applying the Implicit Function Theorem to $f_\theta(x_0 + t^*(\theta)\omega) = 0$ yields the exact analytic gradient at the final hit point $p = x_0 + t^*\omega$:
$$\frac{\partial t^*}{\partial \theta} = -\left(\nabla_p f_\theta(p) \cdot \omega\right)^{-1} \frac{\partial f_\theta(p)}{\partial \theta}, \qquad \frac{\partial p}{\partial \theta} = -\omega\left(\nabla_p f_\theta(p) \cdot \omega\right)^{-1} \frac{\partial f_\theta(p)}{\partial \theta}$$
  - **Step 3 (Normal-Aware Appearance & Loss):** Shading network $g_\phi(p, \omega, n)$ must take the surface normal $n = \nabla_p f_\theta(p)$ to model shiny reflections and pass appearance gradients into geometry. Trained with $\mathcal{L}_{\text{photo}} + \lambda_{\text{eik}}\big|\|\nabla f_\theta\| - 1\big| + \mathcal{L}_{\text{mask}}$.
  - **Limitations of Pure Surface Rendering:** Only updates 1 point per ray; discontinuous at occlusion boundaries/silhouettes ($\nabla_p f \cdot \omega \to 0$); **requires a foreground mask loss** ($\min_t f_\theta(x_0 + t\omega)$) to pull missed rays toward the object.
- **Unifying Surfaces and Volume Rendering (VolSDF & NeuS):**
  - **Idea:** Predict an SDF $d = f_\theta(x)$, convert d into a volume density σ via a smooth analytical formula, and use standard NeRF ray marching (smooth gradients along the entire ray, **no masks required**).
  - **VolSDF:** $\sigma(x) = \alpha\Psi_\beta(-d_\Omega(x))$, where Ψ_β is the CDF of a zero-mean Laplace distribution with sharpness β ($\sigma \to \alpha$ inside, 0 outside, $\alpha/2$ on the surface).
  - **Why Naive Bell-Shaped SDF-to-Density Fails (Bias):** If you set density to a symmetric bell curve $\sigma(t) = \phi_s(f(p(t)))$ peaking at the surface $f = 0$, transmittance $T(t) = \exp(-\int_0^t \sigma)$ is already decaying as the ray climbs the bell curve. Thus the rendering weight $w(t) = T(t)\sigma(t)$ satisfies $w'(t^*) = -T\sigma^2 < 0$, meaning $w(t)$ **peaks strictly in front of the true surface** (systematic depth bias!).
  - **NeuS Unbiased Density:** Designs the opaque density $\rho(t) = \max\left(\frac{-\frac{d\Phi_s}{dt}(f(p(t)))}{\Phi_s(f(p(t)))}, 0\right)$ where $\Phi_s(x) = (1 + e^{-sx})^{-1}$ is the sigmoid and $\phi_s = \Phi_s'$. Then along a planar surface crossing, $T(t) = \Phi_s(f(p(t)))$ and the weight $w(t) = T(t)\rho(t) = |\nabla f \cdot \omega|\,\phi_s(f(p(t)))$ **peaks at the exact zero-crossing** $f = 0$.

Lecture 10: Differentiable Primitive Rendering & 3D Gaussian Splatting (3DGS)

Core Concept: Volume ray marching wastes queries in empty space. Primitive rendering projects explicit primitives onto the screen (rasterization: scatter) and blends softened footprints front-to-back for real-time (>130 FPS) differentiable rendering.

- **Differentiable Mesh Rendering (SoftRas / PyTorch3D):**
  - **Why Hard Mesh Rendering Is Not Differentiable:** Ray-triangle barycentrics (α, β) are differentiable w.r.t. vertices of the visible triangle, but pixel color is a step function at **1) silhouette/occlusion boundaries** (moving a vertex across a pixel center causes a sudden jump) and **2) depth-ordering flips** (z-buffer $\arg\min$ has zero gradient for hidden triangles).
  - **Two-Part Fix:** 1) *Make triangles soft* in 2D screen space: $m_k = \text{sigmoid}(-\Delta_k/\sigma)$ where Δ_k is signed pixel distance to triangle k's edge ($\Delta < 0$ inside). 2) *Aggregate the K closest triangles* via depth-softmax weights $w_k \propto m_k e^{-\lambda_k/\gamma}$ so partially occluded triangles still receive depth gradients.
  - **Ray Tracing vs. Rasterization Loop Order:** Ray tracing loops 'for pixel: for triangle' ($\Theta(PN)$ tests, redundant for nearby pixels). Rasterization swaps the loops to 'for triangle: project to 2D bbox and update pixel K-buffers', reducing work to $\Theta(N\bar{f} + PK)$.
- **3D Gaussian Splatting (3DGS — Kerbl et al.):**
  - **The Primitive (59 floats):** Mean $\mu \in \mathbb{R}^3$ (3), scale $s \in \mathbb{R}^3$ (3), rotation quaternion $q \in \mathbb{R}^4$ (4), opacity $\alpha \in [0, 1]$ (1), and degree-3 Spherical Harmonics color coefficients ($(3+1)^2 \times 3 = 48$).
  - **Why Factorize Covariance $\Sigma = RSS^\top R^\top$:** Taking raw gradient steps on Σ can yield negative eigenvalues (invalid Gaussian). Factorizing into rotation R and diagonal scale S guarantees $\Sigma = (RS)(RS)^\top$ is **always positive semi-definite**.
  - **EWA Splatting Projection to 2D:** World-to-camera is affine ($\Sigma_c = W\Sigma W^\top$), which keeps Gaussians exact. Perspective division $\pi(X, Y, Z) = (f_x X/Z, f_y Y/Z)$ is nonlinear, so 3DGS linearizes π around the Gaussian mean μ_c using its Jacobian $J = \frac{\partial \pi}{\partial x_c}\big|_{\mu_c}$:

$$\mu_{2D} = \pi(W\mu + t), \qquad \Sigma_{2D} = JW\Sigma W^\top J^\top \in \mathbb{R}^{2\times 2}$$

  - **Fast Tile Rasterizer:** 1) Split screen into 16×16 pixel tiles; instantiate a Gaussian copy for each overlapped tile with a 64-bit key '[tile ID (32b) | depth (32b)]'. 2) Run **one global GPU Radix Sort** to get front-to-back lists for all tiles simultaneously. 3) Blend front-to-back per pixel: $C = \sum_k c_k \bar{\alpha}_k T_k$ with $\bar{\alpha}_k = \alpha_k \exp(-\frac{1}{2} d^\top \Sigma_{2D}^{-1} d)$ (clamped ≤ 0.99, skipped if $< 1/255$, early-stopped when $T < 10^{-4}$).
  - **Backward Pass Trick:** Walks **back-to-front**, recovering $T_i = T_{i+1}/(1 - \bar{\alpha}_i)$ on the fly from the stored final transmittance T_{final} in $\mathcal{O}(1)$ memory per pixel: $\frac{\partial C}{\partial \bar{\alpha}_i} = T_i c_i - \frac{1}{1-\bar{\alpha}_i} \sum_{j>i} c_j \bar{\alpha}_j T_j$.
  - **Initialization & Adaptive Density Control:** Initialized from sparse SfM points (random init degrades background). Every 100 steps, Gaussians with large view-space positional gradients (meaning they fail to fit local geometry) are either **Cloned** (if small: under-reconstruction) or **Split** with scale $\div 1.6$ (if large: over-reconstruction). Low-opacity ($\alpha < 0.005$) Gaussians are pruned, and α is periodically reset to remove floaters. Anisotropy is crucial (isotropic balls fail on thin leaves/spokes).
  - **Limitations & 2DGS:** 3DGS has high memory ($\sim$ 700 MB), **popping artifacts** (sorts once per Gaussian center rather than per pixel), and lacks exact surface normals because 3D ellipsoids have view-dependent cross-sections. **2D Gaussian Splatting (2DGS)** flattens primitives into planar 2D disks ($S = \text{diag}(s_u, s_v, 0)$) with exact normals and closed-form ray-plane intersection.

Lecture 11: Relightable 3D Representations (Inverse Rendering)

Core Concept: Standard NeRF and 3DGS bake capture lighting into outgoing color $g_\phi(x, \omega)$ and **cannot be relit**. Relighting requires inverting the Rendering Equation to disentangle *Geometry*, *Material (BRDF)*, and *Light*.

- **The Rendering Equation & The Fundamental Ambiguity:**

$$L_o(x, \omega_o) = \int_\Omega \underbrace{f_r(x, \omega_i, \omega_o)}_{\text{Material (BRDF)}} \underbrace{L_i(x, \omega_i)}_{\text{Incident Light}} \underbrace{(n^\top \omega_i)}_{\text{Geometry}} d\omega_i$$

  Why Inverse Rendering Fails Unconstrained: A capture under one illumination only observes the product Material×Light. Many factorizations produce identical training pixels (e.g., red paint under white light = white paint under red light; or baking all shadows and shading directly into the diffuse albedo $a(x)$ under a flat white environment light) yet fail catastrophically under a new light.
- **Remedy 1 — Constrain the Physical Model (NeRS, PhySG, IRGS):**
  - **Light & Material Constraints:** Assume distant environment light $L_i(x, \omega) \approx L_e(\omega)$ (often monochrome/gray to lock chromaticity into albedo) and split BRDF into spatially varying diffuse albedo $a(x)/\pi$ plus a low-parameter shared specular roughness lobe $f_s(\omega_i, \omega_o; \rho)$.

  - **PhySG (Closed-Form Integration):** Represents the environment map, specular BRDF lobe, and cosine term as **Spherical Gaussians (SGs)** $G(\omega) = \mu e^{\lambda(\xi^\top \omega - 1)}$. Because the product of two SGs is another SG and the integral of an SG over the sphere has a closed form $\mu \frac{2\pi}{\lambda}(1 - e^{-2\lambda})$, PhySG integrates the rendering equation in closed form without Monte Carlo sampling. **Flaw:** Ignores visibility $V(x, \omega_i)$, so cast shadows get baked into dark albedo patches!
  - **IRGS (Relightable 2D Gaussians with Shadows):** Stores normal, albedo, and roughness on 2D Gaussians. First splats/blends attributes into a per-pixel G-buffer $(\bar{n}, \bar{a}, \bar{r}, \bar{d})$ (*blend first, then shade once per pixel*), then traces 256 shadow rays per pixel through the Gaussians to obtain visibility $V(x, \omega_j) = \prod(1 - \alpha_i)$ and 1-bounce indirect color $L_{\text{ind}}(x, \omega_j)$.
- **Remedy 2 — Capture More Informative Data (Light Stage / OLAT):**
  - **Linearity of Light Transport:** Photons do not interact, so $I(c_A A + c_B B) = c_A I_A + c_B I_B$ (in linear HDR space). Capturing **One Light at a Time (OLAT)** images I_k builds the light-transport matrix $T = [I_1 \mid \cdots \mid I_K]$; relighting under any new environment ℓ is a simple weighted sum $I_{\text{new}} = \sum_k \ell_k I_k$ with **zero BRDF or geometry estimation** (requires a static subject in a light stage).
- **Remedy 3 — Learned Data Priors (MaterialFusion, Neural Gaffer):**
  - **MaterialFusion:** Adds a diffusion prior loss $\gamma \mathcal{L}_{\text{prior}}(\text{albedo}_\theta, \text{roughness}_\theta)$ inside per-scene physical optimization to penalize baked shading in the albedo.
  - **Neural Gaffer:** Diffusion U-Net that relights 2D images directly without explicit BRDF/geometry. Crucially, the target HDR environment map is **rotated into the camera coordinate frame** (giving every pixel a consistent directional meaning) and fed as both an LDR tone-mapped copy (visible look) and a log-normalized copy (high dynamic range peaks).

Lecture 12: Gaussian-Based Active Planning & Active Mapping

Core Concept: In Active Perception, the robot chooses its next camera pose T_{t+1} to maximize task information gain under a view budget. Replacing each 3D Gaussian's RGB color c_k with a task utility vector s_k turns the fast 3DGS rasterizer into an **occlusion-aware viewpoint scoring engine**: $g(T, \tau) = \sum_{u \in \Omega} \sum_k w_k(u; T)\, s_k(\tau, T)$.

- **Coverage-Driven Baseline (MAGICIAN):**
  - Predicts unobserved geometry with a 3D Volume Occupancy Network to place **Imagined Gaussians** where opacity $\alpha_k = \hat{p}_{\text{occ}}$ and color $c_k = \text{novelty} \in \{0, 1\}$ ($1 =$ unseen, $0 =$ seen), then plans trajectories via Beam Search.
  - **Flaws for Robotics:** 1) *Single-glance saturation:* Seeing an object once from 6 m away sets novelty to 0, leaving torn, hole-ridden geometry. 2) *No semantics:* Wastes budget mapping blank walls instead of tabletop objects.
- **Semantic Object-Centric Dual-Map Architecture:**
  - **1) Online Gaussian Map (Final Output & Renderer):** Fuses 2D OneFormer semantic predictions using a memory-efficient **4-Slot Depth-Gated Voting** scheme. A Gaussian g_k only receives a semantic vote at pixel u if it passes the **Visible Surface Gate** $|z_k(T_t) - D_t(u)| \le \delta_{\text{surf}}$ (prevents labels from bleeding onto occluded walls behind foreground objects).
  - **2) 5 cm Voxel Planning State (Built from Depth Alone):** Tracks 'free / occupied / unknown' space (for collision-free flight and **pruning mid-air Gaussian floaters**), frontier flags, closest viewing distance z_v, 24-bit directional bitmasks $(b_{\text{seen}}, b_{\text{feas}})$, and majority-voted Gaussian semantic class $c(v)$.
- **Multi-Signal Channel Rendering & Horizon-5 Beam Search:**
  - **Three Voxel Signals Painted onto Gaussians:** 1) **Discovery (s_{disc}):** Active on frontier voxels viewed from their open side (higher when closer). 2) **Coverage (s_{cov}):** Active on surface voxels seen from < 2 directions when viewed from a new feasible sector. 3) **Objects (s_{obj}):** Active on unretired foreground objects ($c(v) \in \mathcal{C}_{\text{obj}}$); rewards each **halving of viewing distance** down to 1 m (4 m $\to 2$ m $\to 1$ m) plus new viewing sectors until the object is **retired** ($s_{\text{obj}} = 0$).
  - **Beam Search ($W = 5$ sequences, $H = 5$ steps ahead):** Expands neighbor poses, renders the multi-channel Gaussian utility map, discounts surfaces already seen earlier in the candidate trajectory to avoid double-counting, and executes the first step of the best sequence (receding horizon).
  - **Task Modularity (Object Navigation):** Swapping the painted channels to (*Frontier*, *Out-of-sight Doorway Relay*, and *VLM Target Bounding Box*) repurposes the exact same renderer and beam-search planner for zero-shot object navigation.